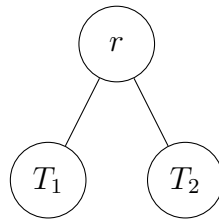


6 Alberi

6.1 Alberi Binari di Ricerca

Definizione induttiva

- $\emptyset$ è un albero;
- Se r è un nodo, T_1 e T_2 alberi $\Rightarrow r(T_1, T_2)$ è un albero.



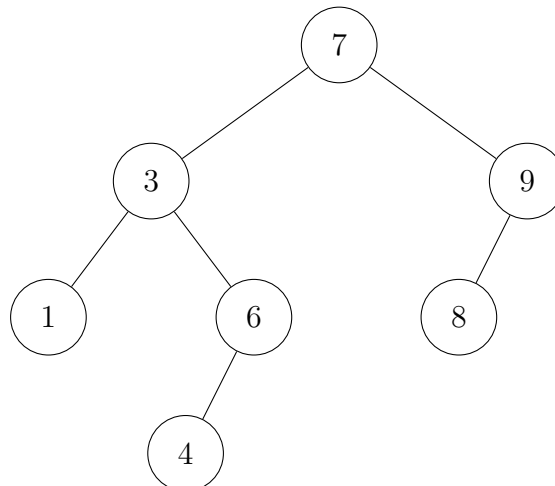
Ogni nodo x ha i seguenti campi:

- $x.p$
- $x.key$
- $x.left$
- $x.right$

Proprietà $\forall r$

- $\rightarrow$ Per ogni nodo y in T_1 $y.key \leq x.key$;
- $\rightarrow$ Per ogni nodo y in T_2 $y.key \geq x.key$.

Esempio Ecco un albero binario di ricerca d'esempio:



6.1.1 Visita Simmetrica

La visita simmetrica (ordine infisso) visita i nodi in ordine crescente.

IN-ORDER(x)

```

1  if  $x \neq \text{NIL}$ 
2      IN-ORDER( $x.\text{left}$ )
3      PRINT( $x$ ) //  $\Theta(1)$ 
4      IN-ORDER( $x.\text{right}$ )

```

Complessità

$$T(n) = \begin{cases} c & n = 0 \\ T(k) + T(n - k - 1) + d & n > 0, k < n \end{cases}$$

Stima di complessità: $T(n) = (c + d)n + c$.

Vediamo la dimostrazione (per induzione).

($n = 0$) $T(n) = c = (c + d) \cdot 0 + c$

($n \rightarrow n + 1$) $T(n) = T(k) + T(n - k) + d$. Non basta l'induzione ordinaria, usiamo l'**induzione completa**.

($n > 0$) Proprietà vera per $n' < n$

$$\begin{aligned}
 T(n) &= T(k) + T(n - k - 1) + d \\
 &\quad \text{con } T(k) = (c + d)k + c \text{ e } T(n - k - 1) = (c + d)(n - k - 1) + c \\
 &= (c + d)(k + n - k - 1) + 2c + d \\
 &= n(c + d) - c - d + 2c + d \\
 &= n(c + d) + c - \cancel{d} + \cancel{d} \\
 &\cong \Theta(n)
 \end{aligned}$$

6.1.2 Ricerca

Ricerca di una chiave k in un albero radicato nel nodo x .

- Se x è NIL $\Rightarrow$ restituisce NIL;
- Altrimenti se $x.\text{key} = k \Rightarrow$ restituisce x ;
- Altrimenti, ricorre sul prossimo nodo.

SEARCH(x, k)

```
1  if ( $x = \text{NIL}$ ) or ( $x.\text{key} = k$ )
2      return  $x$ 
3  else if  $k < x.\text{key}$ 
4      return SEARCH( $x.\text{left}, k$ )
5  else
6      return SEARCH( $x.\text{right}, k$ )
```

Complessità La complessità è l'altezza h dell'albero, ovvero $O(h)$.

Vediamo una versione iterativa di **Search**.

SEARCH-IT(x, k)

```
1  while ( $x \neq \text{NIL}$ ) or ( $x.\text{key} \neq k$ )
2      if  $k < x.\text{key}$ 
3           $x = x.\text{left}$ 
4      else
5           $x = x.\text{right}$ 
6  return  $x$ 
```

Procedura che restituisce il **minimo** di un albero:

MIN(T)

```
1   $x = T.\text{root}$ 
2  if  $x = \text{NIL}$ 
3      return NIL
4  else
5      while  $x.\text{left} \neq \text{NIL}$ 
6           $x = x.\text{left}$ 
7  return  $x$ 
```

Procedura che restituisce il **massimo** di un albero.

MAX(T)

```
1   $x = T.\text{root}$ 
2  if  $x = \text{NIL}$ 
3      return NIL
4  else
5      while  $x.\text{right} \neq \text{NIL}$ 
6           $x = x.\text{right}$ 
7  return  $x$ 
```

Complessità $O(h)$

6.1.3 Successore

Si intende il nodo elencato dopo un nodo x passato come parametro in una visita simmetrica.

Se le chiavi fossero tutte distinte, allora il **successore** di x è il minimo tra i “nodi più grandi di x ”.

- Se x ha un figlio destro, il **successore** è $\text{MIN}(x.\text{right})$;
- Altrimenti, il successore è l’antenato più vicino di cui x è nel sottoalbero sinistro.

SUCCESSOR(x)

```
1  if  $x.\text{right} \neq \text{NIL}$ 
2      return MIN( $x.\text{right}$ )
3  else
4       $y = x.p$ 
5      while ( $y \neq \text{NIL}$ ) and ( $x = y.\text{right}$ )
6           $x = y$ 
7           $y = y.p$ 
8      return  $y$ 
```

Complessità $O(h)$

6.1.4 Inserimento

INSERT(T, z)

```
1   $x = T.root$ 
2   $y = NIL$ 
3  while  $x \neq NIL$ 
4       $y = x$ 
5      if  $z.key < x.key$ 
6           $x = x.left$ 
7      else
8           $x = x.right$ 
9   $z.p = y$ 
10 if  $y = NIL$ 
11      $T.root = z$ 
12 else
13     if  $z.key < y.key$ 
14          $y.left = z$ 
15     else
16          $y.right = z$ 
```

Complessità $O(h)$

6.1.5 Eliminazione

Distingueremo 2 casi nell'algoritmo:

- (1) z ha al più un figlio;
- (2) z ha due figli.

Per fare ciò, usiamo una funzione ausiliaria **Transplant**, con complessità $O(1)$.

TRANSPLANT(T, u, v)

```
1  if  $u.p = NIL$ 
2       $T.root = v$ 
3  else
4      if  $u = u.p.left$ 
5           $u.p.left = v$ 
6      else
7           $u.p.right = v$ 
8  if  $v \neq NIL$ 
9       $v.p = u.p$ 
```

```
DELETE( $T, z$ )
1  if  $z.left = \text{NIL}$ 
2      TRANSPLANT( $T, z, z.right$ )
3  else if  $z.right = \text{NIL}$ 
4      TRANSPLANT( $T, z, z.left$ )
5  else
6       $y = \text{MIN}(z.right)$ 
7      if  $y.p \neq z$ 
8          TRANSPLANT( $T, y, y.right$ )
9           $y.right = z.right$ 
10          $y.right.p = y$ 
11          $y.left = z.left$ 
12          $y.left.p = y$ 
13         TRANSPLANT( $T, z, y$ )
```

Complessità $O(h)$